CS756 PROJECT-I REPORT

Deep Learning for Detecting AI-Generated Content

Provenance Lab: a two-layer system for essays, source code, images and voice

Submitted by

Sanatan Sharma (CO23355)

Ryanveer Singh (CO23353)

Under the guidance of

Prof. Varun Gupta

October 2026

Abstract

Generative AI now produces essays, source code, photorealistic images and cloned voices that are hard to tell apart from human work, which creates problems for academic integrity, software authorship, misinformation and voice fraud. This project builds Provenance Lab, a single system that detects AI-generated content across these four modalities and explains its decisions.

The system has two layers. The first checks for verifiable provenance: C2PA content credentials, AudioSeal audio watermarks and statistical text watermarks (green-list and SynthID-Text). When no such signal is present, the second layer runs a deep learning detector trained for that modality: BERT with perplexity and stylometric features for essays, CodeBERT with syntax-tree features for code, a Vision Transformer with a frequency-spectrum branch for images, and wav2vec 2.0 speech representations for voice. The text, code and image detectors are trained with binary cross-entropy plus a supervised contrastive loss.

On held-out test data the deployed detectors reach 98.3% on HC3 essays, 85.3% on MAGE essays from 27 AI models, 97.6% on code, 90.7% on images from seven modern generators and 94.3% on modern voice clones. On the same current content the best public open-source detectors reach 69.0%, 62.5% and 78.0% respectively. We also report honest negative findings: accuracy drops on AI models not seen in training, watermarks break under simple edits, and the contrastive loss gave no measurable gain at our scale.

Keywords: AI-generated content detection, deepfake detection, BERT, CodeBERT, Vision Transformer, wav2vec 2.0, contrastive learning, C2PA, watermarking.

Contents

1. Introduction
2. Related work
3. System design
4. Datasets and experimental setup
5. Results
6. The web application
7. Discussion and limitations
8. Conclusion and future work

References

1. Introduction

1.1 Problem

Large language models write fluent essays and working code; diffusion models create photorealistic images; voice-cloning services reproduce a person's voice from seconds of audio. These capabilities are useful, but they also enable academic dishonesty, misrepresented code authorship, image-based misinformation and voice-fraud calls. Reliable, automated detection is therefore an active research problem.

1.2 Gaps in existing detectors

- **Single modality.** Text detectors, image detectors and voice anti-spoofing systems are separate tools with separate interfaces.
- **Poor generalisation.** Detectors learn the quirks of the generators they were trained on and fail on newer ones.
- **Watermarks only help when present.** Provenance standards such as C2PA and SynthID give proof, but most tools do not apply them and simple edits remove them.
- **Over-confident verdicts.** A forced yes/no answer can wrongly accuse a person; few tools express uncertainty.

1.3 Objectives

1. Build one system that handles essays, Python code, images and voice recordings.
2. Combine verifiable provenance (credentials and watermarks) with trained deep learning detectors.
3. Train on public benchmarks with leakage-free splits, and evaluate against simple baselines and popular public detectors.
4. Test honestly on AI models and generators that were not seen in training.
5. Explain every verdict and report uncertainty instead of forcing a decision.

2. Related work

AI-generated text. Early detectors used statistical signals such as perplexity: machine text tends to be more predictable to a language model. Fine-tuned classifiers such as OpenAI's RoBERTa detector followed. Benchmarks such as HC3 (human versus ChatGPT answers) and MAGE (27 language models, 10 domains) showed that detectors trained on one generator degrade on others, and RAID (ACL 2024) showed that detectors are easily fooled by unseen models and adversarial edits.

AI-generated code. CoDET-M4 provides human and LLM-written code across languages and generators. Stylometric and syntax-tree features, together with code language models such as CodeBERT, are the usual approach.

AI-generated images. Generators leave artefacts in the frequency spectrum from upsampling, which CNN and frequency-domain detectors exploit. CIFAKE is a small 32 by 32 benchmark; GenImage covers modern generators including Midjourney and Stable Diffusion at full resolution.

Synthetic speech. The ASVspoof challenges benchmark spoofed speech detection. Self-supervised speech models such as wav2vec 2.0 give strong representations for anti-spoofing.

Provenance and watermarking. C2PA attaches signed content credentials to media. SynthID-Text and green-list watermarks bias token choice so a detector can test for them statistically; AudioSeal embeds an imperceptible watermark in audio.

Contrastive learning. Supervised contrastive learning (Khosla et al., 2020) pulls same-class embeddings together and pushes different classes apart, and has been proposed to improve generalisation.

3. System design

Figure 1 shows the architecture. Every input first passes through the provenance layer. If a verifiable AI signal is found, the system reports it as proof and stops. Otherwise the trained detector for that modality produces a probability, mapped to a verdict: above 70% is AI-generated, below 30% is human, and anything between is reported as uncertain for human review.

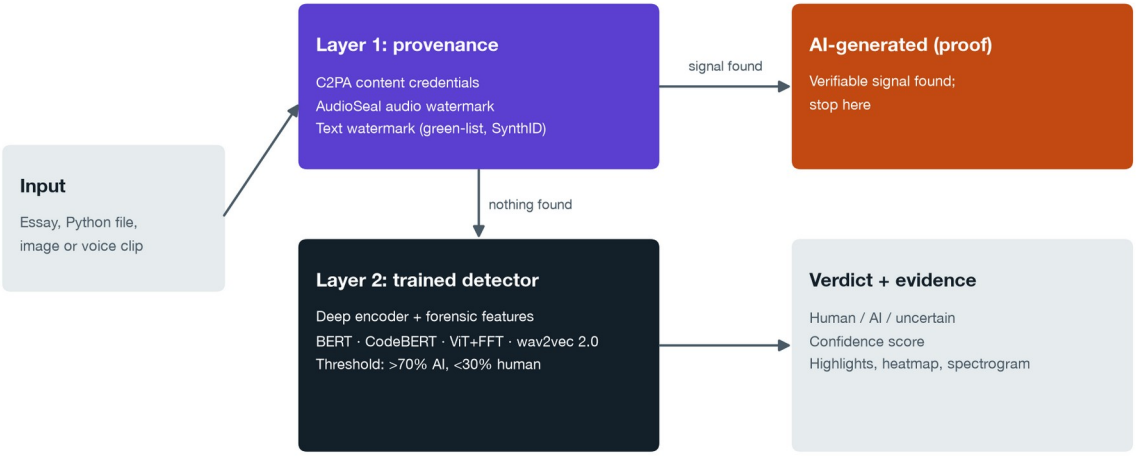


Figure 1. System architecture: provenance first, then a trained detector, then a verdict with evidence.

3.1 Layer 1: provenance

- **C2PA credentials** for images and audio: the system reads the signed manifest and checks whether it declares AI generation (IPTC trainedAlgorithmicMedia).
- **AudioSeal** for voice: a neural watermark detector.
- **Text watermarks** for essays and code: green-list (Kirchenbauer et al.) and SynthID-Text detection by z-score.

3.2 Layer 2: trained detectors

Figure 2 shows the common training pipeline. Each detector fuses a pretrained deep encoder with hand-crafted forensic features through a small fusion network, which feeds a classifier head and a projection head for the contrastive loss.



Figure 2. Training pipeline shared by all detectors.

Content	Deep encoder	Forensic features
Essays	BERT (bert-base-uncased)	Perplexity and its spread under DistilGPT-2, sentence burstiness, vocabulary richness, punctuation and length
Code	CodeBERT	Syntax-tree depth and node mix, function length, identifier style, comment and docstring ratios
Images	ViT-Small at 224 px + FFT-spectrum CNN	Radial power spectrum, high-frequency energy, noise-residual statistics
Voice	wav2vec 2.0 (frozen) + logistic regression	Mean of hidden layers 3, 6, 9 and 12; a spectrogram CNN with MFCC and phase features is kept for comparison

Table 1. Detector design for each modality.

Loss. The text, code and image detectors minimise L = BCE + 0.5 × SupCon (temperature 0.1). The essay model used in the application is additionally trained on diverse data and its decision threshold is calibrated on validation data.

4. Datasets and experimental setup

All experiments use public benchmark datasets downloaded from Hugging Face. Table 2 lists each dataset, what its human and AI classes contain, and how many samples we used for training and testing.

Content	Dataset	Human vs AI	Train / test used
Essays	HC3	Human answers vs ChatGPT	4,000 / 1,000
Essays	MAGE	Human vs 27 LLMs, 14 domains	12,000 (OPT held out) / 1,000
Code	CoDET-M4 (Python)	GitHub, LeetCode, Codeforces vs 5 LLMs	12,000 / 1,000
Images	Tiny-GenImage	ImageNet photos vs 7 generators	14000 / 3500
Images	CIFAKE	CIFAR-10 vs Stable Diffusion (32 px)	6,000 / 2,000
Voice	ASVspoof 2019 LA	Bona fide vs 13 TTS/VC attacks	3,000 / 1,000
Voice	Modern TTS (v4)	YouTube speech vs ElevenLabs, Polly, Hume and others	1339 / 527

Table 2. Public datasets used. All subsets are class-balanced.

4.1 Data hygiene

- Leakage-free splits: by question (HC3), by speaker (ASVspoof), by recording or generation (modern voices), and official splits for code and images.
- Shortcuts removed before reporting: tokenisation spacing and literal "\n" escapes in HC3 answers, leading and trailing silence in ASVspoof, and the JPEG-versus-PNG format difference in GenImage (every image is resized and re-encoded as JPEG).

4.2 Training protocol

- 10% of each training set is held out for validation; training stops when validation loss stops improving for two epochs and the best epoch is restored.
- Pretrained encoders are fine-tuned at a learning rate of 2e-5; new layers at 1e-3; AdamW with weight decay 0.01.
- Hardware: Apple M4 laptop with 16 GB memory, PyTorch on the Metal GPU, with every job run under a memory watchdog.
- Metrics: accuracy, precision, recall, F1 and AUC-ROC, with 95% confidence intervals for comparisons.

5. Results

5.1 Deployed detectors

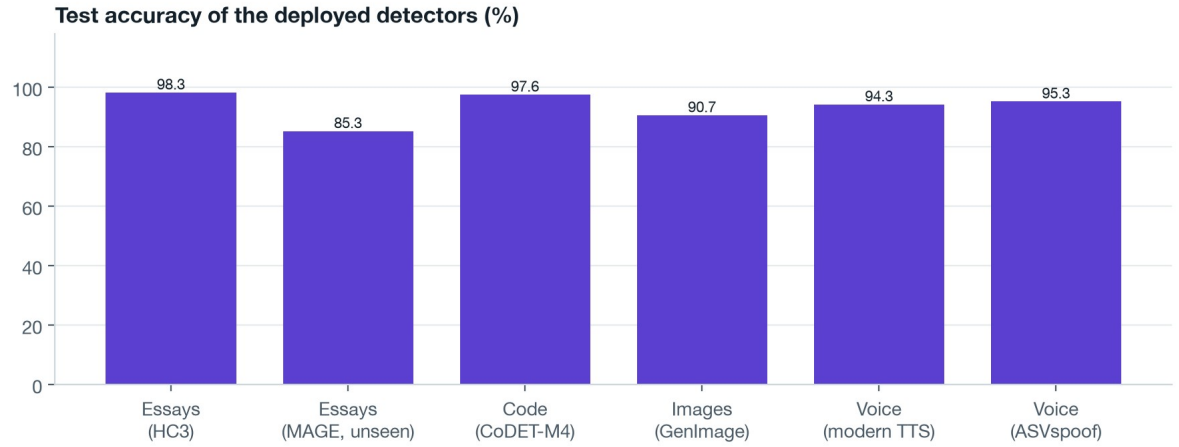
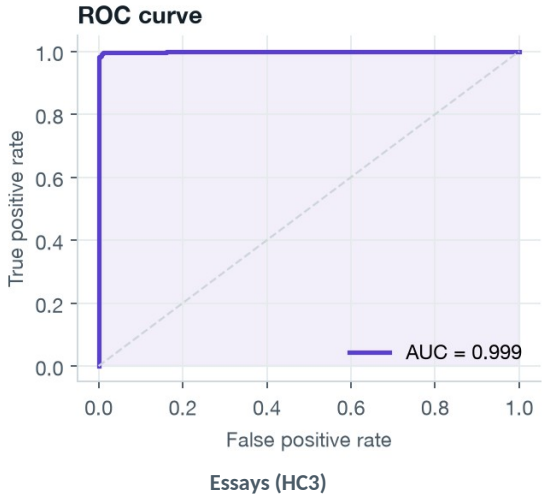


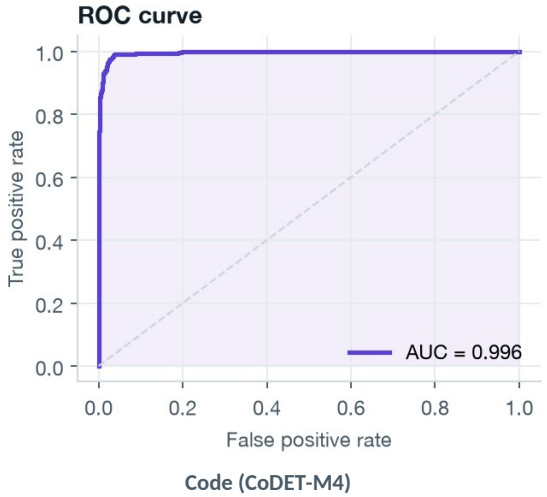
Figure 3. Test accuracy of the detectors used by the application.

Content	Test data	Accuracy	F1	AUC
Essays	HC3	98.3%	98.3%	0.999
Essays	MAGE (unseen models)	85.3%	-	-
Code	CoDET-M4	97.6%	97.6%	0.996
Images	GenImage	90.7%	90.6%	0.971
Voice	Modern TTS	94.3%	-	-
Voice	ASVspoof 2019	95.3%	-	-

Table 3. Test metrics of the deployed detectors.



Essays (HC3)



Code (CoDET-M4)

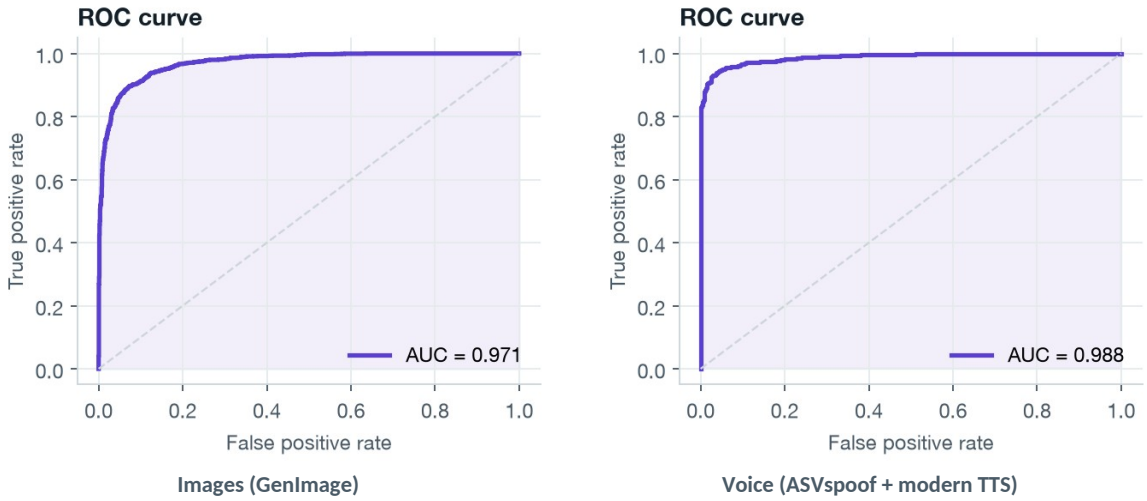


Figure 4. ROC curves on the test sets.

5.2 Learning curves: neither overfitting nor underfitting

Figure 5 plots training and validation loss and accuracy for each detector. Validation curves that track the training curves, with the best epoch restored, show that the models learned general patterns rather than memorising the training set.

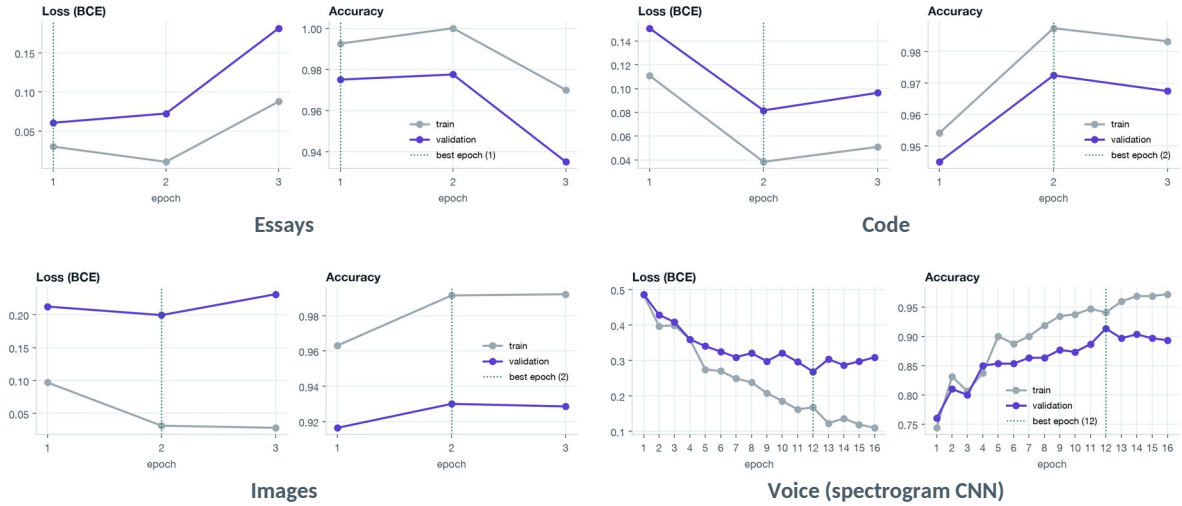


Figure 5. Training and validation loss and accuracy per epoch; the dotted line marks the epoch kept. Train-test accuracy gaps: essays 1.0, code 1.2, images 8.5 points.

5.3 Comparison with public detectors

Popular open-source detectors from Hugging Face were run unchanged on exactly our test sets. On current AI content, where no public model was trained on the test set, our detectors lead in every modality (Figure 5).

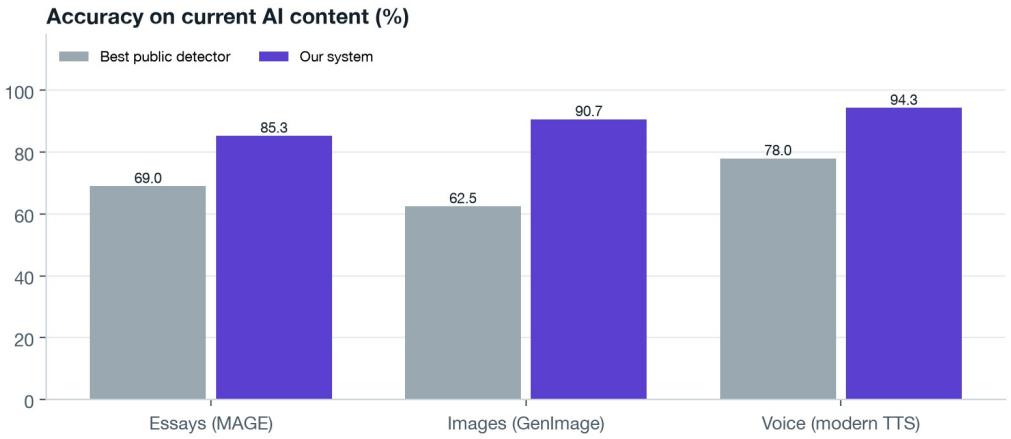


Figure 6. Accuracy on current AI content: best public detector versus our system.

Content	Public detector	Benchmark set	Current set
Essays	roberta-base-openai-detector	HC3 92.8%	MAGE 65.5%
Essays	roberta-base-ai-text-detection-v1	HC3 90.2%	MAGE 69.0%
Essays	chatgpt-detector-roberta	HC3 89.5%	MAGE 53.9%
Images	AI-image-detector	CIFAKE 59.5%	GenImage 58.1%
Images	ai_vs_real_image_detection	CIFAKE 98.4%	GenImage 56.4%
Images	ai-vs-human-image-detector	CIFAKE 40.6%	GenImage 62.5%
Voice	Deepfake-audio-detection	ASVspoof 53.4%	Modern 78.0%
Voice	Deepfake-audio-detection-V2	ASVspoof 72.2%	Modern 51.8%

Table 4. Public detectors on our test sets. dima806 was trained on CIFAKE and the HC3 ChatGPT detector on HC3, so they have a home advantage on those benchmark sets.

Commercial tools such as GPTZero and Turnitin are closed and paid, so we rely on independent evaluations: Weber-Wulff et al. (2023) found 14 tools, Turnitin included, "neither accurate nor reliable", and RAID (Dugan et al., 2024) found detectors easily fooled by unseen models and simple edits.

5.4 Comparison with simple baselines

To check that the deep models earn their complexity, we trained well-known simple methods on exactly the same splits (Table 5).

Content	Best simple method	Simple	Ours
Essays (HC3)	Forensic features + logistic regression	98.4%	98.3%
Code	TF-IDF character n-grams + logistic regression	93.1%	97.6%
Images (CIFAKE)	Forensic features + logistic regression	82.8%	97.2%
Voice (ASVspoof)	Forensic features + logistic regression	76.0%	96.3%

Table 5. Simple baselines trained and tested on the same splits.

The deep models clearly beat simple methods on code, images and voice. On HC3 a simple feature-based classifier already matches the deep model: HC3 is an easy benchmark, so the deep model's value for essays shows on unseen AI models instead (Section 5.5).

5.5 Generalisation to unseen AI models

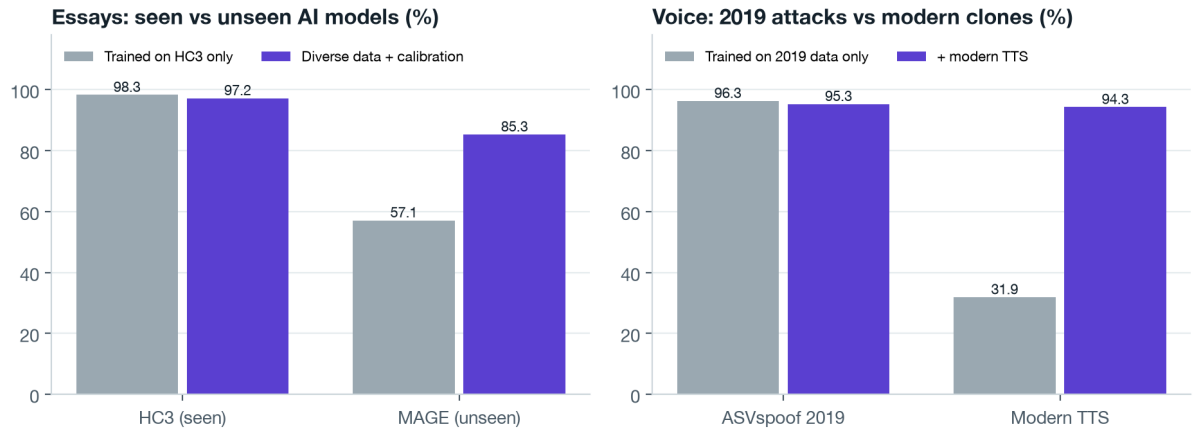


Figure 7. Left: essays trained only on HC3 collapse on MAGE; diverse data and calibration recover much of the gap. Right: a voice model trained on 2019 attacks fails on modern clones until modern TTS is added to training.

Trained only on HC3, the essay detector scores 98.3% on HC3 but 57.1% on MAGE. Training on HC3 plus a diverse MAGE training subset (with the OPT model family held out entirely) and calibrating the threshold on MAGE validation data gives 85.3% on MAGE while 86.2% of human texts are still cleared. On the held-out OPT family, 77.2% of AI texts are caught.

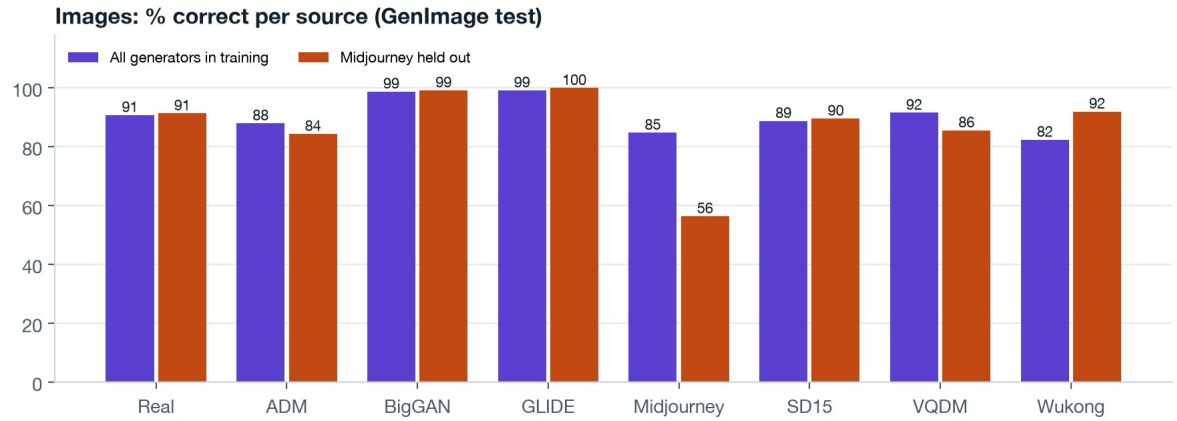


Figure 8. Images: accuracy per source. With Midjourney removed from training, 56.4% of Midjourney images are still caught, against 84.8% when included.

Code shows a similar pattern: with all GPT-written code removed from training, the detector still catches 78.0% to 85.0% of GPT code, against 99.0% when GPT is included.

5.6 Effect of the contrastive loss

We trained each detector twice with identical data and settings, once with and once without the supervised contrastive loss (Table 6).

Content	Cross-entropy only	With SupCon	Difference (pts, 95% CI)
Essays (HC3)	98.9%	98.3%	-0.6 ± 1.0
Code (4k)	94.4%	95.5%	+1.1 ± 1.9
Images (CIFAKE)	97.1%	97.2%	+0.1 ± 1.0
Voice (CNN)	80.0%	80.7%	+0.7 ± 3.5
Code, unseen GPT	85.0%	78.0%	-7.0 ± 10.7

Table 6. Same architecture and data, with and without the supervised contrastive loss.

Every difference lies within its 95% confidence interval, so at our data scale the contrastive loss gave no measurable gain, including on an unseen generator. We report this as a finding. Two other design choices did matter: pretrained speech features beat a spectrogram CNN trained from scratch on voice (80.7% to 96.3%), and training on more varied data improved both code (95.5% to 97.6%) and essays on unseen models.

5.7 Robustness of watermarks

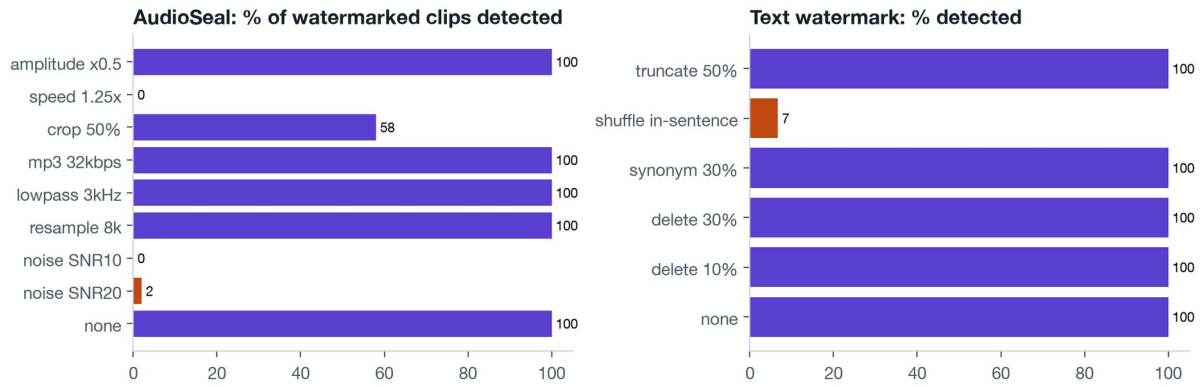


Figure 9. Share of watermarked content still detected after common edits. Orange bars mark edits that break the watermark.

Watermarks are detected every time when content is untouched, and false positives on unwatermarked human content were zero. But added noise or a speed change removes the audio watermark, shuffling words removes text watermarks, and a single JPEG re-save strips C2PA credentials. Provenance is decisive when present but fragile, which is why the trained detectors form the second layer.

6. The web application

The application runs locally (FastAPI back end, custom HTML/CSS/JavaScript front end). A user pastes text or drops a file; the system detects the content type, runs both layers and returns a verdict, a confidence score, the deciding layer and an explanation. Buttons load real held-out test samples and the report marks whether the model was right. An automated end-to-end test of real samples through the API and a headless browser test of every tab completed with no errors.

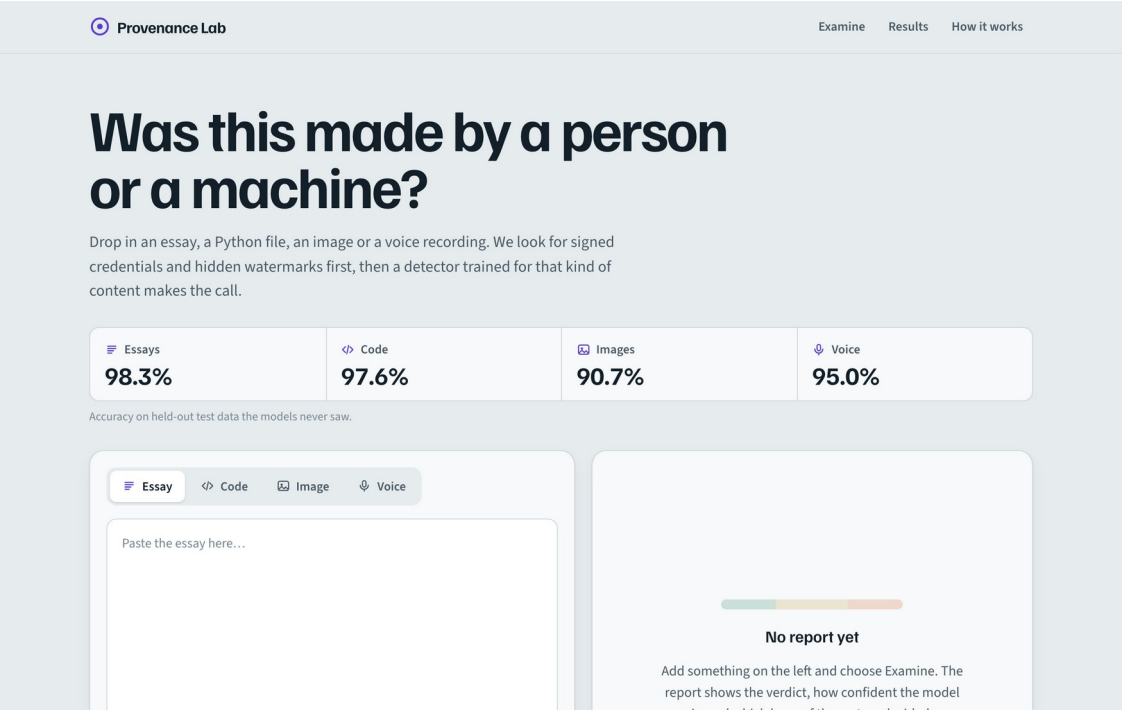


Figure 10. Home screen with the content tabs and the live accuracy of each detector.

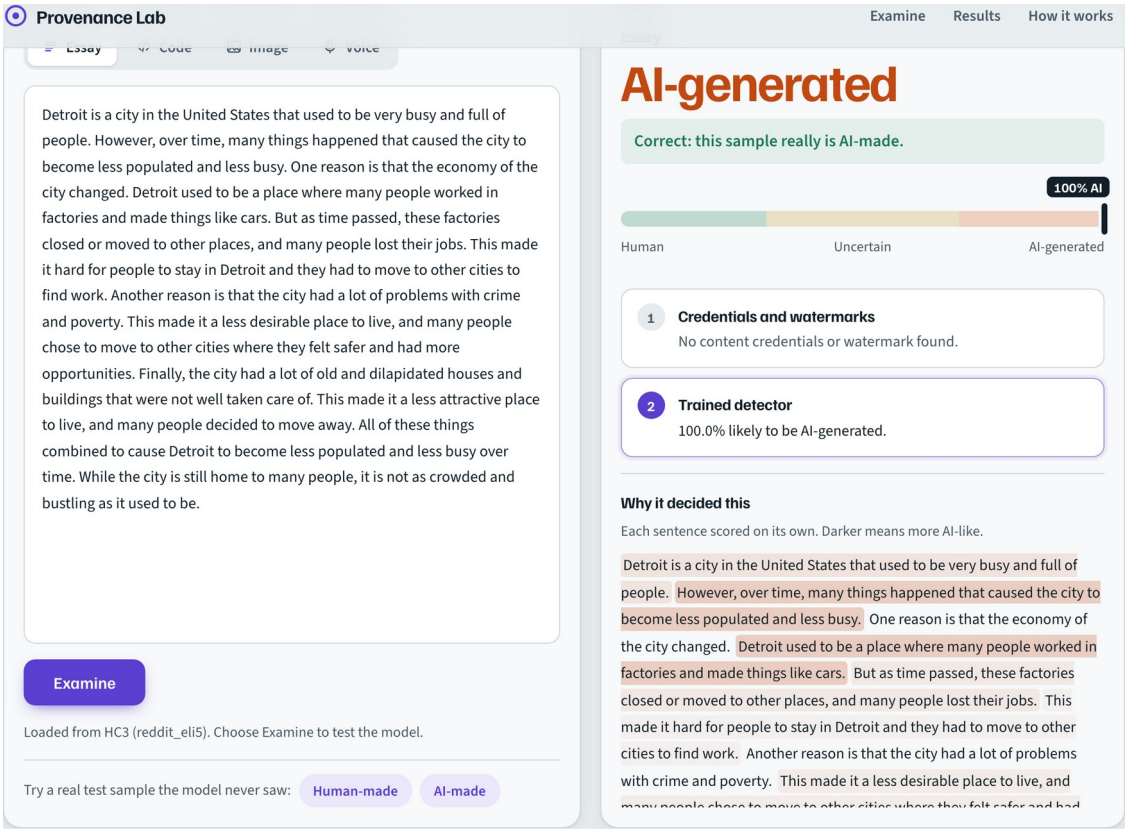


Figure 11. Essay: verdict, confidence scale and per-sentence highlighting (darker means more AI-like).

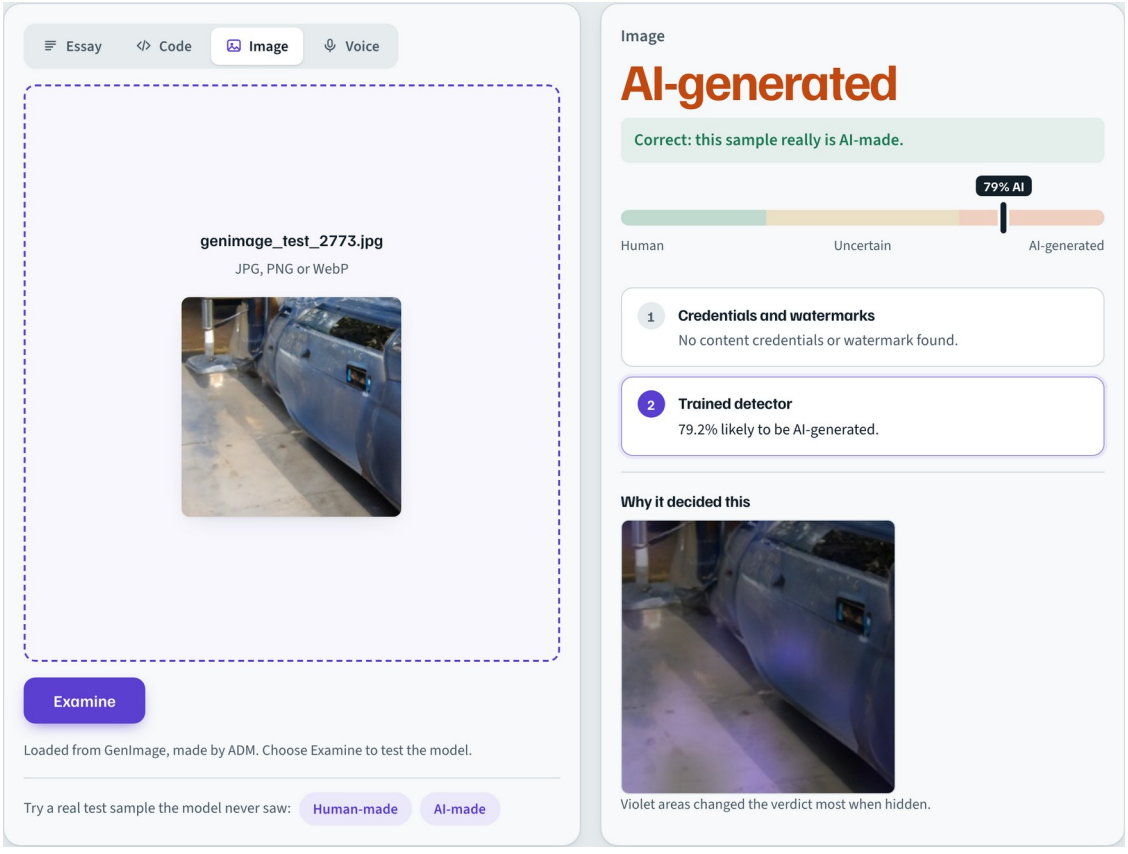


Figure 12. Image: the occlusion heatmap shows the regions that most changed the verdict.

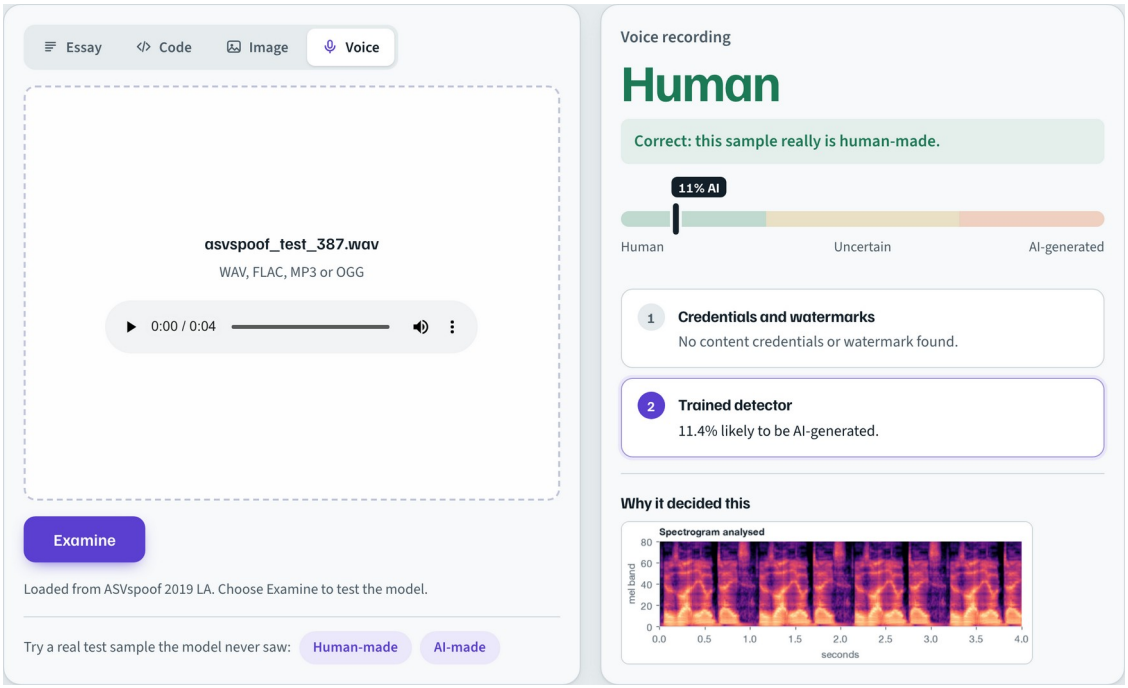


Figure 13. Voice: a real recording correctly cleared, with the analysed spectrogram.

7. Discussion and limitations

- **No detector is perfect.** Benchmark accuracy overstates real-world accuracy: every detector we tested, ours included, loses accuracy on AI models it has not seen. Our response is to train on diverse, current data, calibrate thresholds on validation data, and report uncertainty rather than force a verdict.
- **Training scale.** Models were trained on subsets on a single laptop. Larger and newer data would likely help further.
- **Coverage.** Code detection covers Python only; essays are English; the image detector expects photograph-like content.
- **Provenance access.** Google's SynthID detectors for images and audio are not public; we use open equivalents.
- **Hard cases.** Some ASVspoof attack types and the held-out Midjourney and OPT generators remain difficult.

8. Conclusion and future work

Provenance Lab detects AI-generated essays, code, images and voice in one system, combining verifiable provenance with trained deep learning detectors and explaining every verdict. It reaches 97.6% on code, 90.7% on modern images, 94.3% on modern voice clones and 85.3% on essays from 27 unseen-domain AI models, ahead of every public detector we could run on the same current content. Equally important, the project measures where detection fails and builds that uncertainty into the system.

Future work

- Train on larger and newer datasets (more generators, ASVspoof 5) on a GPU.
- Evaluate and train against paraphrasing, compression and other adversarial edits.
- Extend code detection to more languages and essays to more languages.
- Revisit contrastive learning with many more generators and larger batches.

References

[1] Guo, B. et al. (2023). How Close is ChatGPT to Human Experts? Comparison Corpus, Evaluation, and Detection (HC3). arXiv:2301.07597.

[2] Li, Y. et al. (2024). MAGE: Machine-generated Text Detection in the Wild. ACL 2024.

[3] Orel, D. et al. (2024). CoDET-M4: Detecting Machine-Generated Code in Multi-Lingual, Multi-Generator and Multi-Domain Settings.

[4] Bird, J. J. and Lotfi, A. (2024). CIFAKE: Image Classification and Explainable Identification of AI-Generated Synthetic Images. IEEE Access.

[5] Zhu, M. et al. (2023). GenImage: A Million-Scale Benchmark for Detecting AI-Generated Image. NeurIPS 2023.

[6] Wang, X. et al. (2020). ASVspoof 2019: A Large-Scale Public Database of Synthesized, Converted and Replayed Speech. Computer Speech & Language.

[7] Devlin, J. et al. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. NAACL.

[8] Feng, Z. et al. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages. EMNLP Findings.

[9] Dosovitskiy, A. et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR.

[10] Baevski, A. et al. (2020). wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. NeurIPS.

[11] Khosla, P. et al. (2020). Supervised Contrastive Learning. NeurIPS.

[12] Kirchenbauer, J. et al. (2023). A Watermark for Large Language Models. ICML.

[13] Dathathri, S. et al. (2024). Scalable Watermarking for Identifying Large Language Model Outputs (SynthID-Text). Nature.

[14] San Roman, R. et al. (2024). Proactive Detection of Voice Cloning with Localized Watermarking (AudioSeal). ICML.

[15] Coalition for Content Provenance and Authenticity (C2PA). Technical Specification.

[16] Weber-Wulff, D. et al. (2023). Testing of Detection Tools for AI-Generated Text. International Journal for Educational Integrity, 19, 26.

[17] Dugan, L. et al. (2024). RAID: A Shared Benchmark for Robust Evaluation of Machine-Generated Text Detectors. ACL 2024.